

ACCESO A DATOS

(AD)

lridruejo
Tarea U4
2025 / 2026

ÍNDICE

1.	Introducción.	1
2.	Análisis comparativo.	1
3.	Propuesta técnica.	2
	ESCENARIO A: Aplicación para videojuegos	2
	ESCENARIO B: Sistema de gestión de clientes y productos	3
4.	Conclusiones.	3

1. Introducción.

Los **Sistemas Gestores de Bases de Datos Orientados a Objetos (SGBDOO)** almacenan los datos como objetos, manteniendo la misma estructura que los lenguajes de programación orientados a objetos. Esto evita tener que transformar los datos entre los programas y el almacenamiento.

Los **Sistemas Gestores de Bases de Datos Objeto-Relacionales (SGBDOR)** añaden características orientadas a objetos (como la herencia) al modelo relacional. Siguen almacenando los datos en tablas con filas y columnas como en el modelo relacional al mismo tiempo que ofrecen mejores posibilidades de representación de los datos.

La elección entre un sistema u otro afecta directamente tanto al rendimiento como a la facilidad de desarrollo. Por eso es muy importante seleccionar el enfoque adecuado según las necesidades del proyecto.

2. Análisis comparativo.

	SGBDOO	SGBDOR
Modelado de datos	Ventajas: - Almacenan objetos directamente, sin necesidad de mapeo objeto-relacional (ORM). - Tienen soporte nativo para herencia, polimorfismo y relaciones complejas (jerarquías, grafos). - Son ideales para estructuras de datos anidadas y dinámicas. Desventajas: - Las relaciones entre objetos pueden volverse complejas de gestionar sin un esquema claro. - Tienen un menor soporte para consultas complejas.	Ventajas: - Combinan tablas relacionales con tipos de datos personalizados. - Soportan herencia de tablas y consultas complejas mediante SQL. - Son excelentes para modelar datos estructurados con relaciones definidas (claves foráneas). Desventajas: - Requieren ORM o conversión manual para mapear objetos a tablas. - No son tan buenos para modelar jerarquías profundas.
Escalabilidad	Ventajas: - Proporcionan un alto rendimiento para operaciones de navegación por grafos de objetos complejos. - Son buenos para aplicaciones de tamaño pequeño/mediano con objetos complejos. Desventajas: - Muchos SGBDOO tienen una escalabilidad horizontal limitada. - Pueden presentar cuellos de botella con cargas masivas de usuarios transaccionales.	Ventajas: - Tienen una alta escalabilidad horizontal y vertical, probada en entornos empresariales masivos. - Están optimizados para un alto volumen de transacciones y concurrencia. Desventajas: - Requieren una mayor configuración manual para mantener un rendimiento óptimo. - El coste de licencias puede ser alto.
Facilidad de uso	Ventajas: - Se integran de forma transparente con lenguajes orientados a objetos.	Ventajas: - Cuentan con un amplio ecosistema de herramientas.

	- Realizan las operaciones CRUD directamente sobre los objetos. - No necesitan SQL ni ORM, el proceso de persistencia es casi inapreciable para el desarrollador. Desventajas: - Tienen herramientas de administración y monitorización menos maduras. - Usan lenguajes de consulta propietarios con menor comunidad.	- SQL es un estándar universal con una comunidad enorme y multitud de recursos. - Existen ORMs populares y maduros que facilitan el desarrollo. Desventajas: - Tienen una mayor complejidad inicial por la necesidad de configurar los ORMs. - Requieren de más código para gestionar sesiones y transacciones.
Curva de aprendizaje	Ventajas: - La curva de aprendizaje es más corta para desarrolladores con experiencia en POO. - Hay que aprender menos conceptos nuevos (SQL y ORM). Desventajas: - Requieren aprender APIs específicas. - Existe menos documentación.	Ventajas: - Existe más documentación. Desventajas: - La curva de aprendizaje es más pronunciada porque requiere aprender ORM.

3. Propuesta técnica.

ESCENARIO A: Aplicación para videojuegos

Elegiría un **SGDBOO** por la complejidad de las estructuras de datos. Los personajes, los tokens, el inventario y los mundos son objetos con relaciones complejas y es más fácil y cómodo almacenarlos directamente sin necesidad de convertirlos a tablas relacionales.

Concretamente, elegiría **ObjectDB** porque el desarrollo de db4o se detuvo en 2010. Además, ObjectDB está más optimizado para grafos de objetos profundos y complejos, por lo que las consultas son más eficientes. ObjectDB usa JPA:

```
// CONEXIÓN
EntityManagerFactory emf =
    Persistence.createEntityManagerFactory("database.obj");
EntityManager em = emf.createEntityManager();

// CREATE
em.getTransaction().begin();
em.persist(objeto);
em.getTransaction().commit();

// READ
// Por ID
Objeto obj = em.find(Objeto.class, id);
// Por consulta JPQL
List<Objeto> lista = em.createQuery("SELECT o FROM Objeto o WHERE
    o.propiedad = :valor", Objeto.class).setParameter("valor",
    valor).getResultList();

// UPDATE
```

```
em.getTransaction().begin();
Objeto obj = em.find(Objeto.class, id);
obj.setPropiedad(nuevoValor);
em.getTransaction().commit();

// DELETE
em.getTransaction().begin();
Objeto obj = em.find(Objeto.class, id);
em.remove(obj);
em.getTransaction().commit();

// CERRAR CONEXIÓN
em.close();
emf.close();
```

ESCENARIO B: Sistema de gestión de clientes y productos

Elegiría un **SGDBOR** porque para un SGE, con módulos de ventas y facturación, se requieren garantías ACID sólidas (las operaciones tienen que ser atómicas y consistentes). Para los reportes, son necesarias agregaciones, joins y consultas analíticas. Además, tienen un ecosistema más maduro.

Concretamente, elegiría **PostgreSQL** por coste (es de código abierto y gratuito), modernidad (pero con madurez y una gran comunidad y documentación), y flexibilidad y eficiencia gracias a su soporte nativo de JSONB (mayor facilidad para realizar reportes estadísticos complejos).

```
CONECTAR(URL, USUARIO, CONTRASEÑA)

// CREATE
EJECUTAR("INSERT INTO entidad(campos) VALUES (valores)")

// READ
resultado = EJECUTAR("SELECT campos FROM entidad WHERE campo =
    valor")

// UPDATE
EJECUTAR("UPDATE entidad SET campo = nuevoValor WHERE campo = valor")

// DELETE
EJECUTAR("DELETE FROM entidad WHERE campo = valor")

CERRAR_CONEXIÓN()
```

4. Conclusiones.

Cada sistema tiene sus fortalezas y sus debilidades, y conviene elegir uno u otro en función de las necesidades del proyecto: los **SGBDOO** destacan en proyectos con modelos de datos complejos y jerárquicos, mientras que los **SGBDOR** son ideales para entornos empresariales transaccionales en los que priman la integridad, la escalabilidad, las consultas complejas y la concurrencia.